

Object Oriented

Classes and Objects

- Which of the following is true about class and object?
 - A. Classes are template and objects are instances of the template
 - B. Objects of the same class must have all attributes/data members listed in the class
 - C. object's data value can be accessed through their reference
 - D. Class is abstracted as a combination of important fields and behaviors.
 - E. All of the above
- One of the following is the default value of boolean data type.
 - A. true
 - B. null
 - C. false
 - D. 0
- One of the following is the default value of char data type.
 - A. true
 - B. null
 - C. false
 - D. 0
- One of the following is the correct way of declaring class variable in Java
 - A. static double PI = 3.14;
 - B. double PI = 3.14;
 - C. final double PI=3.14;
 - D. None of the above

- We can use **System** and **String** class in our program without importing the package explicitly. It is because the package is automatically included in any Java program. Which one of the following is this package?

- A. java.util
- B. java.lang
- C. javax.math
- D. java.io
- E. All of the above

- Consider the following string and answer two Questions

String hannah = "Did Hanah see bees? Hanah did.";

What is the value displayed by the expression *hannah.length()*?

- A. 30
- B. 23
- C. 25
- D. None of the above

What is the value returned by the method *hannah.charAt(10)*?

- A. space
- B. E
- C. b
- D. s

- Given the declarations **x**, **y** and **z** are **int**, **float** and **double** variables respectively, which of the following statement is likely to result in error?
- **x = (float)y;**
- **x = (int)z;**
- **y = x;**
- **z = x;**
- **None of the above**

1. One of the following is the output of the following code

```
Public class Output{  
    public static void main(String args[]){  
        int A[];  
  
        int B [] = {10, 20, 30, 40, 50};  
        A = B;  
        A[0] = 0;  
        for(int i = B.length-1; i>=0; i--)  
            System.out.print(B[i]+" ");  
    }  
}
```

- A. 10 20 30 40 50
- B. 0 20 30 40 50
- C. Error
- D. No output
- E. 50 40 30 20 0

- If a method assigns a new value to a **primitive** parameter, will this have any effect on its caller?
 - A. No, because all the method has a copy of its caller's values.
 - B. No, because the method is not allowed to change the value of any parameter.
 - C. Yes, the change will affect the matching value in the caller.
 - D. Yes, because the method and its caller share the same section of memory.
- Which one of the following is false about instance methods?
 - A. can access only static data
 - B. cannot refer to **this** reference
 - C. can only invoke static method
 - D. must be accessed using class name
 - E. all of the above
- One of the following access modifiers allows only the subclasses to access the members.
 - A. public
 - B. private
 - C. package
 - D. default
 - E. protected

- One of the following is wrong about static methods
 - A. It cannot access instance variables
 - B. It can be called (invoked) using object
 - C. It can be called by instance methods
 - D. It can be called by prefixing the class name
 - E. None of the above
- One of the following statements is false about **this** keyword
 - A. **this (...)** is used to call a constructor inside another constructor within the same class
 - B. **this (...)** can be used anywhere within the class
 - C. **this** keyword is used to access instance methods and variables within the same class.
 - D. **this** keyword is used to overcome name space collision
 - E. None of the above
- Which of the following is true about the **finalize()** method
 - A. It will be called before an object is garbage collected
 - B. It can be used to ensure that an object terminates cleanly
 - C. It does not take any parameter and does not return
 - D. Cannot be accessed by objects outside its class
 - E. All of the above

- Consider the following code

```
void start() {  
    A a = new A();  
    B b = new B();  
    a.s(b);  
    b = null; /* Line 5 */  
    a = null; /* Line 6 */  
    System.out.println("start completed"); /* Line 7 */  
}
```

- When is the **B** object, created in line 3, eligible for garbage collection?
 - A. after line 5
 - B. after line 6
 - C. after line 7
 - D. There is no way to be absolutely certain

- One of the following Java codes is correct.

```
A. class T {  
    int a;  
    void test (int a){  
        this.a = a;  
    }  
}
```

```
B. class T {  
    int a;  
    static void test (int a){  
        this.a = a;  
    }  
}
```

```
C. class T {  
    int a;  
    void test (int a){  
        a = this (a);  
    }  
}
```

```
D. class T {  
    int a;  
    void test (int a){  
        this (a);  
    }  
}
```

1. Consider the following code

```
class Exam{
    int a;

    Exam ()
    {
        a = 67;
    }

    Exam (int b)
    {
        a=b;
    }
}
```

```
Exam (Exam ob)
{
    a=ob.a;
}

Public static void main (String
args[]) {
    Exam e1= new Exam();
    Exam e2 = new Exam(90);
    Exam e3 = new Exam(e2);
    e1=e3;
    e2=e1;
}
} //end of the class
```

- Which of the following objects will be collected by the garbage collector?

- A. e2
- B. e3
- C. e1
- D. A and C
- E. None

- A class **Animal** has a subclass **Mammal**. Which of the following is true?
 - A. Because of single inheritance, Mammal can have no subclasses.
 - B. Because of single inheritance, Mammal can have no other parent than Animal.
 - C. Because of single inheritance, Animal can have only one subclass.
 - D. Because of single inheritance, Mammal can have no siblings.
- Which one of the following typecasting statements is correct?
Assume class T is a superclass and class W is a subclass?
 - A. T t = new T();
W w = t;
 - A. W w = new T();
T t = (T)w;
 - A. T t = new W();
W w = (W)t;
 - A. All of the above

- A class *Car* and its subclass *Yaris* have a method *show()* in their class definition. If *compact* refers to an object of type *Yaris*, what will the following code do?

`compact.show();`

- A. calls the `show()` method defined in the class *Yaris*.
 - B. calls the `show()` method defined in the class *Car*.
 - C. The compiler will complain that *show()* has been defined twice.
 - D. Overloading will be used to pick which *show()* is called.
- One of the following is wrong statement if SUP is super class and SUB is the subclass.
 - A. `SUP x = new SUP( )`
 - B. `SUB y = new SUP( )`
 - C. `SUP y = new SUB( )`
 - D. All are correct
 - One of the following is false about **`super()`**.
 - It must be the first line in the constructor
 - Used in the constructor to call superclass's constructor
 - Cannot be used in the user defined method.
 - All of the above
 - None of the above

- What must be true if a child of an abstract parent class does not override all of the parent's abstract methods?
 - A. This is always an error.
 - B. The child class itself must be declared to be abstract.
 - C. Child classes are automatically non-abstract, so this is OK.
 - D. The parent class is in error.
- One of the following classes cannot be inherited
 - A. abstract class
 - B. public class
 - C. final class
 - D. A and C
 - E. B and C
- Which one of the following is false?
 - A. final classes can only be super classes
 - B. final method can be overloaded
 - C. final methods cannot be overridden
 - D. final variable values cannot be changed
 - E. None of the above
- Which of these access specifiers can be used for an interface?
 - A. public
 - B. protected
 - C. private
 - D. All of the mentioned

- One of the following is not true.
 - A. An abstract class must have at least one abstract method
 - B. Final class cannot be subclassed
 - C. Abstract class can be inherited by other classes
 - D. Final class can be instantiated
 - E. None of the above
- Consider the following code and answer the below given question.

```
final class Test{  
    public void display(){  
        System.out.print("This is a test machine");  
    }  
}
```

 - Which one of the following is **WRONG**?
 - A. The method display cannot be overridden
 - B. The class cannot be inherited
 - C. Object cannot be instantiated
 - D. The method display can be overloaded

- What is the output of the following Java program?

```
class X {  
    Y b = new Y();  
    X() {  
        System.out.print("X"); }  
}  
class Y {  
    Y() {  
        System.out.print("Y"); }  
}  
public class Z extends X {  
    Y y = new Y();  
    Z() {  
        System.out.print("Z"); }  
    public static void main(String[] args) {  
        Z z = new Z();  
    }  
}
```

- A. ZY
- B. XZYY
- C. YXZY
- D. YXYZ

- Can an object of a child type be assigned to a variable of the parent type? For example,

```
Card crd;  
BirthDay bd = new BirthDay("Lucinda", 42);  
crd = bd;    // is this correct?
```

 - A. No--there must always be an exact match between the variable and the object types.
 - B. No--but a object of parent type can be assigned to a variable of child type.
 - C. Yes--an object can be assigned to a reference variable of the parent type.
 - D. Yes--any object can be assigned to any reference variable.
- Which one of the following is not a correct statement? (Assume that Person is a super-class and Student is a Sub-class)
 - Person p = new Person();
 - Person p = new Student();
 - Student s = new Person ();
 - All are correct

- Consider the following Java code and answer the below given **Question**.

```
class ParentClass {  
    int x;  
    ParentClass(int a){  
        a = x;  
    }  
    void display(){  
        System.out.println("The value of x: "+ x) ;  
    }  
}
```

- One of the following statements is **FALSE** if you create a class which inherits from the above class.
 - A. Constructor must invoke the parent constructor explicitly
 - B. If the class overrides the display method the access modifier of the overriding method must be more visible
 - C. You must use **super** keyword to access the overridden method in the child class.
 - D. variable x can be accessed without using super keyword if there is no variable with the same name in the child class
 - E. None of the above

- Consider the following class definition and answer the next five **Questions**.

```
class FinalExam extends Exam{
    String str;
    int num;
    void printData( ){
        System.out.println(++num) ;
    }
    FinalExam (String s) {
        str = s;
    }
    FinalExam (int num) {
        super(20) ;
        num = num;
    }
}
```

- One of the following is correct object creation statement
 - A. `FinalExam fe;`
 - B. `FinalExam fe = new FinalExam();`
 - C. `FinalExam fe = new FinalExam("Hello");`
 - D. B and C
- One of the following is the correct way of accessing the method within another class
 - A. `printData()`
 - B. `fe.printData();`
 - C. `FinalExam.printData();`
 - D. All of the above
- What will be the output if the following two statements executed sequentially?

```
FinalExam final = new FinalExam(25);  
final.printData();
```

 - A. 1
 - B. 0
 - C. 25
 - D. None of the above

- One of the following is the setter method for the **str** field in the class
 - A.

```
void setter() {
    str = "Hello";
}
```
 - A.

```
String setStr(Str s){
    str = s;
    return str;
}
```
 - A.

```
String setStr(){
    return str;
}
```
 - A.

```
void setStr(String s) {
    str = s;
}
```
- One of the following casting statements is wrong.
 - A.

```
Exam e = new FinalExam("Welcome");
FinalExam f = (FinalExam)e;
```
 - B.

```
Exam e = new Exam(20);
FinalExam f = (FinalExam)e;
```
 - C.

```
Exam e = new FinalExam(20);
FinalExam f = e;
```
 - D. None of the above

- Which of the following is a correct outline of a method that throws an exception when it finds a problem?

A. `public void someMethod () {
 if (problem)
 throw new Exception("Useful Message");
}`

B. `public void someMethod () throws Exception {
 if (problem)
 Exception("Useful Message");
}`

C. `public void someMethod () throws Exception {
 if (problem)
 throw(new Exception("Useful Message"));
}`

D. `public void someMethod () throws Exception {
 if (problem)
 throw new Exception("Useful Message");
}`